Chapitre 4 : La modularité algorithmique

Hassen NAKBI ::: hassen.nakbi@gmail.com

1ere année Licence

20 novembre 2023

Introduction

Modularité

Lorsque nous travaillons sur des problèmes complexes, il n'est pas rare qu'un algorithme présente des répétitions fréquentes d'actions. Ces répétitions d'actions entraînent souvent des codes d'algorithme volumineux, peu lisibles, et par conséquent, difficiles à entretenir. Pour résoudre ce problème, nous appliquons la modularité, qui consiste à diviser un algorithme en sous-algorithmes plus petits, réutilisables et autonomes, chacun accomplissant une tâche spécifique. Leur utilisation permet de transformer un problème complexe en une série de tâches plus simples.

Introduction

Modularité

La manipulation d'un tableau est considérée comme une tâche complexe, car elle nécessite tout d'abord le remplissage du tableau, suivi du tri dans un ordre spécifique, pour enfin afficher le résultat à l'écran. Ainsi, la modularité consiste à concevoir des modules distincts pour chaque tâche, afin de simplifier et structurer la manipulation du tableau de manière plus efficace.

2 Sous algorithme

- Définition
- Propriétés
- Types
- Paramètres

Définition

Un sous-algorithme est une portion de code qui accomplit une tâche spécifique et peut être utilisée de manière autonome dans un algorithme principal.

2 Sous algorithme

- Définition
- Propriétés
- Types
- Paramètres

Propriétés

- **Indépendance** : Les sous-algorithmes sont des unités indépendantes de code. Ils fonctionnent de manière autonome et n'ont pas besoin de connaître les détails du code appelant.
- **Réutilisabilité** : Les sous-algorithmes sont conçus pour être réutilisables. Vous pouvez les appeler plusieurs fois dans votre programme ou les utiliser dans différents projets.
- **Clarté** : Les sous-algorithmes permettent de rendre le code plus clair et lisible en organisant des tâches complexes en unités plus petites et plus compréhensibles.

2 Sous algorithme

- Définition
- Propriétés
- Types
- Paramètres

Types

Deux types de modules peuvent être utilisés : **fonction et procédure**.

- **Fonction nomfonction**(p1 : type1,p2 : type2):**type de retour**
- **Procédure nomprocédure**(p1 : type1,p2 : type2)

2 Sous algorithme

- Définition
- Propriétés
- Types
- Paramètres

Paramètres

Définition

Les paramètres sont des variables spécifiées dans la déclaration d'une fonction ou d'une procédure et qui permettent de recevoir des données lors de l'appel de celle-ci. Ces données peuvent être utilisées à l'intérieur de la fonction ou de la procédure pour effectuer des opérations spécifiques.

Paramètres

Propriétés

- **Déclaration** : Les paramètres sont déclarés dans l'entête d'une fonction ou d'une procédure. Ils précèdent le nom de la fonction ou de la procédure et sont encadrés par des parenthèses.
- **Types** : Chaque paramètre est associé à un type de données spécifique qui indique le type d'informations qu'il peut recevoir.
- **Passage de valeurs** : Lors de l'appel de la fonction ou de la procédure, les valeurs spécifiques fournies sont appelées arguments et sont assignées aux paramètres correspondants.

Paramètres

Passage par adresse

Fonction somtab(B :TAB,z : entier)

Un paramètre formel désignant un tableau en entrée, modélisé avec un mode de passage par adresse, permet une manipulation efficace et directe des données stockées dans le tableau au sein d'une procédure ou d'une fonction.
Lorsque le tableau est transmis par adresse en tant que paramètre formel, il n'est pas nécessaire de copier l'ensemble des éléments du tableau dans la mémoire locale de la procédure.

3 Les fonctions

- Définition
- Définition algorithmique
- Fonction en langage C
- Application

Les fonctions

Définition

Une fonction en algorithmique est une séquence d'instructions regroupées sous un nom spécifique, destinée à effectuer une tâche particulière. Les fonctions peuvent accepter des données en entrée, appelées paramètres, les traiter, et doivent renvoyer un seul résultat de type simple.

3 Les fonctions

- Définition
- Définition algorithmique
- Fonction en langage C
- Application

Les fonctions

Définition algorithmique

Algorithme principal DEFfonction
VAR VG : type
Fonction **nomfonction**(p1 : type1,p2 : type2):**type de retour**
VAR vl1,vl2 :type
Début
Les instructions de fonction
Renvoyer résultat
Fin
DÉBUT
Appel à une fonction
VG ⟵**nomfonction**(val1,val2)
FIN

3 Les fonctions

- Définition
- Définition algorithmique
- Fonction en langage C
- Application

Fonction en langage C

Programmation C

```
#include <stdio.h>
type VG ;
type de retour nomfonction(type p1, type p2){
type vl1,vl2 ;
Les instructions de fonction ;
return résultat ; }
int main( ) {
Appel à une fonction ;
VG = nomfonction(val1,val2) ;
return 0 ;}
```

3 Les fonctions

- Définition
- Définition algorithmique
- Fonction en langage C
- Application

Fonction moyenne

Fonction moyenne

```
Algorithme principal MoyenneVal
VAR VG : réel
Fonction moyenne(p1,p2 : entier):réel
VAR mr :réel
Début
mr ⟵ (p1 + p2) / 2
Renvoyer mr
Fin
DÉBUT
VG ⟵moyenne(7,11)
Écrire("moyenne : ",VG)
FIN
```

Fonction moyenne

Fonction moyenne

```
#include <stdio.h>
float VG ;
float moyenne(short p1, short p2){
float mr = (p1 + p2) / 2 ;
return mr ; }
int main( ) {
VG = moyenne(7,11) ;
printf("Moyenne : %f",VG) ;
return 0 ;}
```

4 Les procédures

- Définition
- Définition algorithmique
- Procédure en langage C
- Application

Les procédures

Définition

Une procédure en algorithmique est une séquence d'instructions regroupées sous un nom spécifique, permettant d'accomplir une série d'actions sans renvoyer de résultat clair et explicite.

Les procédures sont utilisées pour rendre implicitement à l'algorithme principal des structures de données complexes comme les tableaux et chaînes.

4 Les procédures

- Définition
- Définition algorithmique
- Procédure en langage C
- Application

Les procédures

Définition algorithmique

Algorithme principal DEFprocédure
Procédure **nomprocédure**(p1 : type1, p2 : type2)
VAR vl1,vl2 :type
Début
Les instructions de procédure
Fin
DÉBUT
Appel à une procédure
nomprocédure(val1,val2)
FIN

4 Les procédures

- Définition
- Définition algorithmique
- Procédure en langage C
- Application

Procédure en langage C

Programmation C

```
#include <stdio.h>
void nomprocédure(type p1, type p2){
type vl1,vl2 ;
Les instructions de procédure ;
}
int main( ) {
Appel à une procédure ;
nomprocédure(val1,val2) ;
return 0 ;}
```

4 Les procédures

- Définition
- Définition algorithmique
- Procédure en langage C
- Application

Procédure remplissage

Procédure remplissage

Algorithme principal Remplissage
CONST N = 20
TYPE TAB = tableau[1 .. N] d'entiers
VAR T : TAB

Procédure remplir(A : TAB,
S : entier)
VAR i :entier
Début
Pour i de 1 à S pas ⟵1 faire
Début
Écrire("A[",i,"]=")
Lire(A[i])
FinPour
Fin
DÉBUT
remplir(T,N-10)
FIN

Procédure remplissage

Procédure remplissage

```
#include <stdio.h>
#define N 20
short T[N];
void remplir(short A[ ], short S){
short i;
for(i=0;i<S;i++){
printf("A[%hi]= ",i);
scanf("%hi",&A[i]);} }
int main( ) {
remplir(T,N-10);
return 0;}
```

5 Sous-algorithme récursif

- Définition
- Propriétés
- Fonction récursive
- Procédure récursive en langage C
- Application

Sous-algorithme récursif

Définition

Un sous-algorithme récursif est une séquence d'instructions qui, lors de son exécution, peut faire appel à elle-même pour résoudre une version du problème initial avec une taille réduite.

La récursivité est basée sur la notion de diviser pour régner, où un problème est divisé en sous-problèmes plus petits et la solution finale est construite en combinant les solutions des sous-problèmes.

5 Sous-algorithme récursif

- Définition
- Propriétés
- Fonction récursive
- Procédure récursive en langage C
- Application

Sous-algorithme récursif

Propriétés

Pour appliquer la notion de récursivité dans un sous algorithme, nous devons penser aux notions suivants :

- **Cas de base** : Un sous-algorithme récursif inclut un ou plusieurs cas de base qui spécifient les conditions dans lesquelles l'algorithme doit se terminer sans faire d'autres appels récursifs.
- **Appel récursif** : Un sous-algorithme récursif contient un appel à lui-même dans son propre corps, généralement avec des paramètres modifiés pour représenter une version réduite du problème.

5 Sous-algorithme récursif

- Définition
- Propriétés
- Fonction récursive
- Procédure récursive en langage C
- Application

Fonction récursive

Définition algorithmique

Algorithme principal recursive
VAR VG : type
Fonction **nomfonction**(p1 : type1,p2 : type2):**type de retour**

Début
SI (condition) alors
Renvoyer valeur
SINON
Renvoyer nomfonction(p3,p4)
FinSI
Fin

DÉBUT
VG ⟵**nomfonction**(val1,val2)
FIN

Fonction récursive en langage C

Programmation C

```
#include <stdio.h>
type VG ;
type de retour nomfonction(type p1, type p2){
if (condition)  return valeur ;
else return nomfonction(p3,p4) ;
}
int main( ) {
VG = nomfonction(val1,val2) ;
return 0 ;}
```

Fonction factorielle

```
Algorithme principal Factoriel
VAR F : entier
Fonction factorielle(p : entier):entier
Début
SI (p = 1) alors
Renvoyer 1
SINON
Renvoyer p×factorielle(p-1)
FinSI
Fin
DÉBUT
F ⟵factorielle(7)
Écrire("Résultat : ",F)
FIN
```

Fonction factorielle

Fonction factorielle

```
#include <stdio.h>
long long F ;
long long factorielle(short p){
if (p = = 1) return 1 ;
else return p×factorielle(p-1) ;
}
int main( ) {
F = factorielle(7) ;
printf("Résultat : %lli",F) ;
return 0 ;}
```

Procédure récursive

Définition algorithmique

```
Algorithme principal procrecursive
Procédure nomprocédure(p1 : type1, p2 : type2)
Début
SI (condition) alors
Renvoyer
SINON
nomprocédure(p3,p4)
FinSI
Fin
DÉBUT
nomprocédure(val1,val2)
FIN
```

5 Sous-algorithme récursif

- Définition
- Propriétés
- Fonction récursive
- Procédure récursive en langage C
- Application

Procédure récursive en langage C

Programmation C

```
#include <stdio.h>
void nomprocédure(type p1, type p2){
type vl1,vl2 ;
if (condition)  return ;
else nomprocédure(p3,p4) ;
}
int main( ) {
procédure(val1,val2) ;
return 0 ;}
```

5 Sous-algorithme récursif

- Définition
- Propriétés
- Fonction récursive
- Procédure récursive en langage C
- Application

Procédure remplissage

Procédure remplissage

Algorithme principal Remplissage
CONST N = 20
TYPE TAB = tableau[1 .. N] d'entiers
VAR T : TAB

Procédure remplir(A : TAB,
S,i : entier)
Début
SI (i = S+1) alors
Renvoyer
SINON
Écrire("A[",i,"]=")
Lire(A[i])
remplir(A,S,i+1)
FinSI
Fin
DÉBUT
remplir(T,N-10,1)
FIN

Procédure remplissage

Procédure remplissage

```
#include <stdio.h>
#define N 20
short T[N];
void remplir(short A[ ], short S,short i){
if (i == S) return ;
else { printf("A[%hi]= ",i) ;
scanf("%hi",&A[i]) ;
remplir(A,N-10,i+1) ;}
}
int main( ) {
remplir(T,N-10,0) ;
return 0 ;}
```